RESOLVER LA ACTIVIDAD QUE SE PLANTEA EN EL DOCUMENTO ACTIVIDAD_5.5

1. **Una práctica entregada exactamente igual que la corrección del profesor, se evaluará con una nota de 6.**
2. **Se evaluará la práctica expresada en las propias palabras del alumno.**
3. **Se evaluará la corrección ortográfica y que las frases estén bien construidas. En general se evaluará la legibilidad del documento. 4. Se evaluará la presentación del documento.**

5. Se penalizarán dos prácticas iguales, tanto en presentación, como en contenido.

6. SE PODRÁ SOLICITAR QUE EL ALUMNO/A DEFIENDA SU TRABAJO EN PÚBLICO

RÚBRICA DE EVALUACIÓN

0 – 6 Puntos: la solución al problema de la práctica. 0 puntos no está resuelta. 6 puntos la práctica está resuelta correctamente.

0 – 0.5 Puntos: el formato del fichero ha de ser PDF. 0 puntos el formato del fichero no es PDF. 1 puntos el formato del fichero es PDF.

0 – 1 Puntos: formato de presentación. 0 la presentación es mala. 1.5 la presentación es muy buena y aporta referencias bibliográficas adecuadamente.

0 – 1.5 Puntos: expresión y corrección lingüística. 0 no se expresa adecuadamente y comete faltas de ortografía. 1.5 se expresa correctamente y con sus propias palabras y no comete faltas de ortografía.

0 – 1 Puntos. La presentación del documento es excelente, y además de expresarse correctamente, con sus propias palabras y no comete faltas de ortografía, aporta información adicional relevante con el tema del ejercicio, sumando valor al conjunto total del documento.

Enunciado de la Actividad:

ACTIVIDAD 5.5

Haz un programa que cree una nueva sede, dos departamentos para esta nueva sede y dos empleados para cada uno de estos departamentos. Verifica que los datos se crean correctamente comprobando el contenido de las tablas creando un método en el service llamado “verificarDatosCreados” y que sea llamado desde el runner. Comprueba también los datos conectándote a un cliente MySQL.

Sube el código desarrollado a tu perfil en GitHub y adjunta el enlace en el documento anterior.

Respuesta:

En esta actividad se ha desarrollado un programa con Hibernate y Spring Boot que crea una nueva sede, dos departamentos asociados a esa sede y dos empleados para cada uno de los departamentos.

Para ello, he reutilizado la estructura creada en la actividad anterior, basada en entidades JPA, repositorios JpaRepository, un servicio para la lógica de negocio y un CommandLineRunner para lanzar el proceso al arrancar la aplicación.

```java
package com.acc.datos.hibernate_project.servicios;

import com.acc.datos.hibernate_project.pojos.Departamento;
import com.acc.datos.hibernate_project.pojos.Empleado;
import com.acc.datos.hibernate_project.pojos.Sede;
import com.acc.datos.hibernate_project.repositorios.DepartamentoRepository;
import com.acc.datos.hibernate_project.repositorios.EmpleadoRepository;
import com.acc.datos.hibernate_project.repositorios.SedeRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service  2 usages  new *
public class SetupInicialService {

    @Autowired
    private SedeRepository sedeRepository;

    @Autowired
    private DepartamentoRepository departamentoRepository;

    @Autowired
    private EmpleadoRepository empleadoRepository;

    @Transactional  1 usage  new *
    public void crearDatosDeEjemplo() {
        System.out.println("--- CREANDO DATOS DE LA ACTIVIDAD 5.5 ---");

        // Crear nueva sede
        Sede nuevaSede = new Sede();
        nuevaSede.setNomSede("Barcelona Norte");
        Sede sedeGuardada = sedeRepository.saveAndFlush(nuevaSede);

        System.out.println("Sede creada con ID: " + sedeGuardada.getIdSede());

        // Crear dos departamentos para esa sede
        Departamento deptoVentas = new Departamento();
        deptoVentas.setNomDepto("Ventas");
```

```java
16      public class SetupInicialService {
28          public void crearDatosDeEjemplo() {
40              deptoVentas.setNomDepto("Ventas");
41              deptoVentas.setSede(sedeGuardada);
42              Departamento deptoVentasGuardado = departamentoRepository.saveAndFlush(deptoVentas);
43
44              System.out.println("Departamento creado con ID: " + deptoVentasGuardado.getIdDepto());
45
46              Departamento deptoRRHH = new Departamento();
47              deptoRRHH.setNomDepto("Recursos Humanos");
48              deptoRRHH.setSede(sedeGuardada);
49              Departamento deptoRRHHGuardado = departamentoRepository.saveAndFlush(deptoRRHH);
50
51              System.out.println("Departamento creado con ID: " + deptoRRHHGuardado.getIdDepto());
52
53              // Crear dos empleados para Ventas
54              Empleado emp1 = new Empleado();
55              emp1.setDni("11111111A");
56              emp1.setNomEmp("Carlos Perez");
57              emp1.setDepartamento(deptoVentasGuardado);
58              empleadoRepository.save(emp1);
59
60              Empleado emp2 = new Empleado();
61              emp2.setDni("22222222B");
62              emp2.setNomEmp("Lucia Gomez");
63              emp2.setDepartamento(deptoVentasGuardado);
64              empleadoRepository.save(emp2);
65
66              // Crear dos empleados para RRHH
67              Empleado emp3 = new Empleado();
68              emp3.setDni("33333333C");
69              emp3.setNomEmp("Marta Ruiz");
70              emp3.setDepartamento(deptoRRHHGuardado);
71              empleadoRepository.save(emp3);
72
73              Empleado emp4 = new Empleado();
74              emp4.setDni("44444444D");
75              emp4.setNomEmp("Javier Lopez");
76              emp4.setDepartamento(deptoRRHHGuardado);
77              empleadoRepository.save(emp4);
```

```java
        System.out.println("--- DATOS CREADOS CORRECTAMENTE ---");
    }

    @Transactional(readOnly = true)  no usages  new *
    public void verificarDatosCreados() {
        System.out.println("--- VERIFICANDO DATOS CREADOS ---");

        List<Sede> sedes = sedeRepository.findAll();
        System.out.println("Total de sedes: " + sedes.size());
        for (Sede sede : sedes) {
            System.out.println("Sede -> ID: " + sede.getIdSede() + ", Nombre: " + sede.getNomSede());
        }

        List<Departamento> departamentos = departamentoRepository.findAll();
        System.out.println("Total de departamentos: " + departamentos.size());
        for (Departamento depto : departamentos) {
            System.out.println("Departamento -> ID: " + depto.getIdDepto()
                    + ", Nombre: " + depto.getNomDepto()
                    + ", ID Sede: " + depto.getSede().getIdSede());
        }

        List<Empleado> empleados = empleadoRepository.findAll();
        System.out.println("Total de empleados: " + empleados.size());
        for (Empleado emp : empleados) {
            System.out.println("Empleado -> DNI: " + emp.getDni()
                    + ", Nombre: " + emp.getNomEmp()
                    + ", ID Depto: " + emp.getDepartamento().getIdDepto());
        }

        System.out.println("--- FIN DE LA VERIFICACIÓN ---");
    }
}
```

En el método crearDatosDeEjemplo se insertan los nuevos registros de forma transaccional. Después, se ha implementado el método verificarDatosCreados, que consulta los datos almacenados en las tablas y los muestra por consola para comprobar que se han persistido correctamente.

```java
package com.acc.datos.hibernate_project.util;

import com.acc.datos.hibernate_project.servicios.SetupInicialService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.stereotype.Component;

@Component  new *
public class AppRunner implements CommandLineRunner {

    @Autowired
    private SetupInicialService setupInicialService;

    @Override  new *
    public void run(String... args) throws Exception {
        System.out.println("=========================================================");
        System.out.println("APLICACIÓN ARRANCADA. EJECUTANDO LÓGICA DE LA ACTIVIDAD 5.5...");
        System.out.println("=========================================================");

        try {
            setupInicialService.crearDatosDeEjemplo();
            setupInicialService.verificarDatosCreados();
        } catch (DataIntegrityViolationException e) {
            System.err.println("ERROR DE INTEGRIDAD DE DATOS: No se pudieron guardar los datos.");
            System.err.println("Causa: " + e.getMostSpecificCause().getMessage());
            System.err.println("Acción: La transacción ha sido deshecha (ROLLBACK) por Spring.");
        }

        System.out.println("=========================================================");
        System.out.println("LÓGICA FINALIZADA. La aplicación está lista.");
        System.out.println("=========================================================");
    }
}
```

Finalmente, también se ha verificado el resultado conectándose a MySQL Workbench y consultando directamente las tablas sede, departamento y empleado, comprobando que las claves foráneas quedan correctamente relacionadas.

Copy/paste de la ejecución:

```
=========================================================
APLICACIÓN ARRANCADA. EJECUTANDO LÓGICA DE LA ACTIVIDAD 5.5...
=========================================================
--- CREANDO DATOS DE LA ACTIVIDAD 5.5 ---
Hibernate:
    insert
    into
        sede
        (nom_sede)
```

```
    values
        (?)
Sede creada con ID: 2
Hibernate:
    insert
    into
        departamento
        (nom_depto, id_sede)
    values
        (?, ?)
Departamento creado con ID: 2
Hibernate:
    insert
    into
        departamento
        (nom_depto, id_sede)
    values
        (?, ?)
Departamento creado con ID: 3
Hibernate:
    select
        e1_0.dni,
        e1_0.id_depto,
        d1_0.id_depto,
        d1_0.nom_depto,
        d1_0.id_sede,
        s1_0.id_sede,
        s1_0.nom_sede,
        e1_0.nom_emp
    from
        empleado e1_0
    join
```

```
        departamento d1_0
            on d1_0.id_depto=e1_0.id_depto
    join
        sede s1_0
            on s1_0.id_sede=d1_0.id_sede
    where
        e1_0.dni=?
Hibernate:
    select
        e1_0.dni,
        e1_0.id_depto,
        d1_0.id_depto,
        d1_0.nom_depto,
        d1_0.id_sede,
        s1_0.id_sede,
        s1_0.nom_sede,
        e1_0.nom_emp
    from
        empleado e1_0
    join
        departamento d1_0
            on d1_0.id_depto=e1_0.id_depto
    join
        sede s1_0
            on s1_0.id_sede=d1_0.id_sede
    where
        e1_0.dni=?
Hibernate:
    select
        e1_0.dni,
        e1_0.id_depto,
        d1_0.id_depto,
```

```
        d1_0.nom_depto,
        d1_0.id_sede,
        s1_0.id_sede,
        s1_0.nom_sede,
        e1_0.nom_emp
    from
        empleado e1_0
    join
        departamento d1_0
            on d1_0.id_depto=e1_0.id_depto
    join
        sede s1_0
            on s1_0.id_sede=d1_0.id_sede
    where
        e1_0.dni=?
Hibernate:
    select
        e1_0.dni,
        e1_0.id_depto,
        d1_0.id_depto,
        d1_0.nom_depto,
        d1_0.id_sede,
        s1_0.id_sede,
        s1_0.nom_sede,
        e1_0.nom_emp
    from
        empleado e1_0
    join
        departamento d1_0
            on d1_0.id_depto=e1_0.id_depto
    join
        sede s1_0
```

```
        on s1_0.id_sede=d1_0.id_sede
    where
        e1_0.dni=?
--- DATOS CREADOS CORRECTAMENTE ---
Hibernate:
    insert
    into
        empleado
        (id_depto, nom_emp, dni)
    values
        (?, ?, ?)
Hibernate:
    insert
    into
        empleado
        (id_depto, nom_emp, dni)
    values
        (?, ?, ?)
Hibernate:
    insert
    into
        empleado
        (id_depto, nom_emp, dni)
    values
        (?, ?, ?)
Hibernate:
    insert
    into
        empleado
        (id_depto, nom_emp, dni)
    values
        (?, ?, ?)
```

```
--- VERIFICANDO DATOS CREADOS ---
Hibernate:
    select
        s1_0.id_sede,
        s1_0.nom_sede
    from
        sede s1_0
Total de sedes: 2
Sede -> ID: 1, Nombre: Madrid Central
Sede -> ID: 2, Nombre: Barcelona Norte
Hibernate:
    select
        d1_0.id_depto,
        d1_0.nom_depto,
        d1_0.id_sede
    from
        departamento d1_0
Total de departamentos: 3
Departamento -> ID: 1, Nombre: Tecnologias de la Informacion, ID Sede: 1
Departamento -> ID: 2, Nombre: Ventas, ID Sede: 2
Departamento -> ID: 3, Nombre: Recursos Humanos, ID Sede: 2
Hibernate:
    select
        e1_0.dni,
        e1_0.id_depto,
        e1_0.nom_emp
    from
        empleado e1_0
Total de empleados: 5
Empleado -> DNI: 11111111A, Nombre: Carlos Perez, ID Depto: 2
Empleado -> DNI: 12345678A, Nombre: Ana Lopez, ID Depto: 1
Empleado -> DNI: 22222222B, Nombre: Lucia Gomez, ID Depto: 2
```

Empleado -> DNI: 33333333C, Nombre: Marta Ruiz, ID Depto: 3

Empleado -> DNI: 44444444D, Nombre: Javier Lopez, ID Depto: 3

--- FIN DE LA VERIFICACIÓN ---

==========

LÓGICA FINALIZADA. La aplicación está lista.

==========

Capturas ejecución y Base de datos:

```
=======================================================
APLICACIÓN ARRANCADA. EJECUTANDO LÓGICA DE LA ACTIVIDAD 5.5...
=======================================================
--- CREANDO DATOS DE LA ACTIVIDAD 5.5 ---
Hibernate:
    insert
    into
        sede
        (nom_sede)
    values
        (?)
Sede creada con ID: 2
Hibernate:
    insert
    into
        departamento
        (nom_depto, id_sede)
    values
        (?, ?)
Departamento creado con ID: 2
Hibernate:
    insert
    into
        departamento
        (nom_depto, id_sede)
    values
        (?, ?)
Departamento creado con ID: 3
Hibernate:
    select
        e1_0.dni,
        e1_0.id_depto,
        d1_0.id_depto,
        d1_0.nom_depto,
        d1_0.id_sede,
        s1_0.id_sede,
        s1_0.nom_sede,
```

```
        sede s1_0
Total de sedes: 2
Sede -> ID: 1, Nombre: Madrid Central
Sede -> ID: 2, Nombre: Barcelona Norte
Hibernate:
    select
        d1_0.id_depto,
        d1_0.nom_depto,
        d1_0.id_sede
    from
        departamento d1_0
Total de departamentos: 3
Departamento -> ID: 1, Nombre: Tecnologias de la Informacion, ID Sede: 1
Departamento -> ID: 2, Nombre: Ventas, ID Sede: 2
Departamento -> ID: 3, Nombre: Recursos Humanos, ID Sede: 2
Hibernate:
    select
        e1_0.dni,
        e1_0.id_depto,
        e1_0.nom_emp
    from
        empleado e1_0
Total de empleados: 5
Empleado -> DNI: 11111111A, Nombre: Carlos Perez, ID Depto: 2
Empleado -> DNI: 12345678A, Nombre: Ana Lopez, ID Depto: 1
Empleado -> DNI: 22222222B, Nombre: Lucia Gomez, ID Depto: 2
Empleado -> DNI: 33333333C, Nombre: Marta Ruiz, ID Depto: 3
Empleado -> DNI: 44444444D, Nombre: Javier Lopez, ID Depto: 3
--- FIN DE LA VERIFICACIÓN ---
====================================================
LÓGICA FINALIZADA. La aplicación está lista.
====================================================
2026-04-14T13:20:46.844+02:00  INFO 1976 --- [hibernate-project] [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'default'
2026-04-14T13:20:46.849+02:00  INFO 1976 --- [hibernate-project] [ionShutdownHook] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Shutdown initiated...
2026-04-14T13:20:46.877+02:00  INFO 1976 --- [hibernate-project] [ionShutdownHook] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Shutdown completed.

Process finished with exit code 0
```

Sede:

```
1  SELECT * FROM sede;
2  SELECT * FROM departamento;
3  SELECT * FROM empleado;
```

Result Grid | Filter Rows: | Edit:

id_sede	nom_sede
1	Madrid Central
2	Barcelona Norte
NULL	NULL

Departamento:

```
SELECT * FROM sede;
SELECT * FROM departamento;
SELECT * FROM empleado;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id_depto	nom_depto	id_sede
1	Tecnologias de la Informacion	1
2	Ventas	2
3	Recursos Humanos	2
NULL	NULL	NULL

Empleado:

```
SELECT * FROM sede;
SELECT * FROM departamento;
SELECT * FROM empleado;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

dni	nom_emp	id_depto
11111111A	Carlos Perez	2
12345678A	Ana Lopez	1
22222222B	Lucia Gomez	2
33333333C	Marta Ruiz	3
44444444D	Javier Lopez	3
NULL	NULL	NULL

Link a GitHub:

https://github.com/DGuelar/Practicas/tree/main/T5/hibernate-project/src/main/java/com/acc/datos/hibernate_project